

UNIT-I

Syllabus

- Linux Utilities
- File handling utilities,
- Security by file permissions,
- Process utilities,
- Disk utilities,
- Networking commands,
- Filters, Text processing utilities and Backup utilities,
- sed – scripts, operation, addresses, commands,
- applications, awk – execution, fields and records, scripts, operation,
- patterns, using system commands in awk.

Introduction

History of UNIX:

- UNIX is an Operating System (OS).
- Developed in 1969 at AT&T Bell Labs by Ken Thompson and Dennis Ritchie.
- Originally designed as a command-line, time-sharing system for mini-computers.
- Predecessor of Linux and many modern OS variants.

History of LINUX:

- LINUX was created by Linus Torvalds in 1991.
- It is an open-source operating system.
- Derived from UNIX, and shares many of its core principles and architecture.

Why LINUX/UNIX?

- ❖ Free and Open Source
- ❖ Customizable
- ❖ Highly Stable
- ❖ Secure Architecture
- ❖ Portable
- ❖ Written in C

File handling Utilities :

- Tools to manage files and directories in Linux.
- Used for **creating, viewing, copying, moving, and deleting** files and directories..
- Commands: ls, cp, mv, rm, mkdir, rmdir, touch

1. ls : List Directory Contents

Example: ls -l (or) ls -l /home/usha

File Creation

Common Text Editors in Linux

1. nano (Beginner friendly, terminal-based)	▪ nano filename.txt ▪ Save file: Press CTRL + O, then Enter. ▪ Exit: Press CTRL + X.
2. vi / vim (Powerful, widely used)	▪ vi filename.txt ▪ Insert mode → for typing text (press i to enter insert mode) Save and exit: :w → save :q → quit :wq → save and quit :q! → quit without saving
3. cat	▪ cat > filename ▪ Save file: CTRL+D ▪ Append file: cat >> filename

2. CP Command in Linux:

The cp command is used to copy files and directories from one location to another.

Copy a single file to a new file or directory.	cp [OPTIONS] SOURCE DEST	cp file1.txt file2.txt cp file1.txt /backup/
Copy multiple files into a directory.	cp [OPTIONS] SOURCE... DIRECTORY	cp file1.txt file2.txt file3.txt /backup/
Explicitly specify the target directory for the source files.	cp [OPTIONS] --target-directory=DIRECTORY SOURCE...	cp --target-directory=/backup/ file1.txt file2.txt

Options	What it Does	Easy Meaning
-a	Archive mode	Copy everything exactly as it is
--backup	Make backup before overwrite	Keep the old version
-b	Simple backup	Add ~ to old file name
-f	Force overwrite	Delete and copy if needed

3. mv – Move or Rename Files

Syntax: `$ mv [source] [destination] (or)`

`$ mv [OPTIONS] filename/directory filename/directory`

Ex1: `mv report.txt archive/`

Ex2: `mv oldname.txt newname.txt`

OPTIONS : `-f` = forcefully move the file, don't ask questions.

`-i` = interactively move the file, ask before overwriting.

☐ Move multiple files to a directory : `mv [file1] [file2] ... [destination_directory]`

Ex: `mv file1.txt tmp/file2.txt newdir`

4. rm – Remove Files or Directories

Syntax : rm [options] [file or directory]

Examples:

1) Delete a single file (or) multiple files	rm file1.txt (or) rm file1.txt file2.txt
2) Delete a directory and its contents recursively	rm -r myfolder
3) Force delete files without any prompts	rm -f file1.txt file2.txt
4) Interactive delete	rm -i *.txt
5) To delete a directory tree:	rm -ir tmp

mkdir Command in Linux:

The mkdir (make directory) command is used to create new directories.

Syntax: mkdir [options] directory_name

1) -m (mode): Set access permissions while creating the directory

Ex: mkdir -m 755 mydir

2) -p (parents): Create parent directories as needed

Ex: mkdir -p projects/web/html

3) -v (verbose): Show messages for each created directory

mkdir -v myfolder

In COMMAND: ln command is used to create link to a file (or) directory.

SYNTAX: ln [options] existingfile(or directory)name newfile(or directory)name

OPTIONS:

OPTIONS	Use	Syntax	Example
(-s) : Symbolic link (Soft link)	• A symbolic link is like a shortcut • It just points to the original file . • it's not a copy.	ln -s existingfile linkname	ln -s notes.txt shortcut.txt
(-i) : Interactive link	Asks for confirmation before overwriting a file if the link name already exists.	ln -i existingfile linkname	ln -i report.txt link.txt
(-f): Force link	Does not prompt before overwriting the destination link.	ln -f existingfile linkname	ln -f data.txt link.txt

Example4: Search for a file name test.txt and its permissions are 775 in a given box

```
find / -perm 775 -name test.txt
```

Example5: How about searching files with SUID bit set and file permissions are 755?

```
find / -perm 4755
```

Example6: Search for all the files with name test.txt and the owner of this file is amar

```
find / -user Surendra -name test.txt
```

Example7: find all the files whos name is test.txt and owned by a group called redcluster

```
find / -group redcluster -name test.txt
```

Pwd COMMAND:

Short for print working directory the pwd command displays the name of the current working directory.

Syntax

```
pwd
```

Examples: pwd

```
/home/computerhope/public_html
```

Security by file permissions :

Unix operating system supports **three types of users**. They are,

User Types: 1. Owner 2. Group 3.Others

File Access Modes: r – Read, w – Write, x – Execute

Change Permissions Syntax: chmod [permissions] filename

Example: \$ chmod a+rw newfile

Give execute permission to the user only:	\$chmod u+x script.sh
Remove write permission from group:	\$ chmod g-w report.txt
Add read and execute permissions to all users:	\$ chmod a+rx myfile
Remove all permissions from others:	\$ chmod o-rwx confidential.txt
Set permissions: user = read/write, group = read, others = none	\$ chmod u=rw,g=r,o= file1

Numeric Mode Examples:

Permission	Binary	Octal
rwX	111	7
rw-	110	6
r--	100	4
r-x	101	5
--x	001	1

Examples:

- `$ chmod 744 program.c` // user: rwX (7), group: r-- (4), others: r-- (4)
- `$ chmod 666 data.txt` // Allow read/write for all users
- `$ chmod 755 run.sh` // Make a file executable by everyone
- `$ chmod 700 private.sh` // Give only the owner read, write, and execute permissions:

Security by file permissions

Number	Octal Permission Representation	Ref
0	No permission	---
1	Execute permission	--X
2	Write permission	-W-
3	Execute and write permission: 1 (execute) + 2 (write) = 3	-wX
4	Read permission	r--
5	Read and execute permission: 4 (read) + 1 (execute) = 5	r-X
6	Read and write permission: 4 (read) + 2 (write) = 6	rw-
7	All permissions: 4 (read) + 2 (write) + 1 (execute) = 7	rwX

Security by file permissions

Here's an example using testfile. Running `ls -l` on testfile shows that the file's permissions are as follows:

```
$ls -l testfile
```

```
-rwxrwxr-- 1 root  users 1024 Nov 2 00:10 testfile
```

Then each example `chmod` command from the preceding table is run on testfile, followed by `ls -l` so you can see the permission changes:

```
$ chmod 755 testfile
```

```
$ls -l testfile
```

```
-rwxr-xr-x 1 root  users 1024 Nov 2 00:10 testfile
```

```
$chmod 743 testfile
```

```
$ls -l testfile
```

```
-rwxr---wx 1 root  users 1024 Nov 2 00:10 testfile
```

```
$chmod 043 testfile
```

Security by file permissions

Changing Owners and Groups:

While creating an account on Unix, it assigns a owner ID and a group ID to each user. All the permissions mentioned above are also assigned based on Owner and Groups.

Two commands are available to change the owner and the group of files:

chown: The chown command stands for "change owner" and is used to change the owner of a file.

chgrp: The chgrp command stands for "change group" and is used to change the group of file.

Changing Ownership:

The chown command changes the ownership of a file. The basic syntax is as follows:

```
$ chown [new_owner] [file_name]
```

The value of user can be either the name of a user on the system or the user id (uid) of a user on the system.

Following example:

```
$ chown root1 testfile
```

Changes the owner of the given file to the user **root1**

NOTE: The super user, root, has the unrestricted capability to change the ownership of a any file but normal users can change only the owner of files they own.

Security by file permissions

Changing Group Ownership:

- The chgrp command changes the group ownership of a file. The basic syntax is as follows:

```
$ chgrp [new_group] [file_name]
```

- The value of group can be the name of a group on the system or the group ID (GID) of a group on the system.
- Following example: \$ chgrp special testfile
- Changes the group of the given file to **special** group.

Process Utilities

These are used to **monitor, manage, and terminate** processes in Linux.

1. ps(Process Status): Displays the current running processes.

Syntax: ps [options]

Options: -e → All processes

-f → Full-format listing

-u user → Processes owned by user

-l → Long listing

Example: ps -f

UID	PID	PPID	C	STIME	TTY	TIME	CMD
user	2540	2536	0	15:31	pts/1	00:00:00	bash
user	2639	2540	0	15:51	pts/1	00:00:00	ps -f

2. who: Shows users currently logged into the system.

Syntax: who [options]

Options: -H → Display column headings

-u → Show detailed list

Example: who -uH

NAME	LINE	TIME	IDLE	PID	COMMENT
usha	tty1	Aug 4 11:08	.	4578	

3. w: Displays who is logged in and what they are doing.

Syntax: w

14:08:30 up 2:25, 3 users, load average: 0.00, 0.01, 0.05

USER	TTY	FROM	LOGIN@	IDLE	JCPU	PCPU	WHAT
usha	tty1	172.16.10.5	11:08	1:00	0.10s	0.10s	bash

4. kill: Terminates a process using its PID.

Syntax: kill -signal PID

Common Option: -9 to force kill

Example: kill -9 19426

5. pkill: Kills processes by name (instead of PID).

Syntax: pkill processname

Disk Utilities

Used for monitoring **disk usage and space availability**.

1. df (Disk Free): Displays available disk space.

Syntax: df -h

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/

Explanation: -h gives human-readable sizes.

Shows how much space is used and available on each mounted filesystem.

2. du(Disk Usage): Shows space used by files and directories.

Syntax: du [options] [file/directory]

Options: -a → Show all files

-b → Show in bytes

-c → Grand total

-h → Human readable

-s → Summary (no subdirectories)

Example: du -ah ~/Documents

Mathematica

4.0K ./resume.pdf

8.0K ./project/report.docx

12K ./project

20K .

Networking Commands

- The commands which we will use for the communication between more than one system for sharing resources or any other issue are known as the networking commands.
- Commands such as telnet, ftp, rlogin, finger are commonly used networking

1. telnet: telnet is a internet protocol which enables user to connect a remote system and transfer data.

Syntax: telnet hostname_or_ip

Example: telnet 192.168.1.10

Trying 192.168.1.10...

Connected to 192.168.1.10.

Escape character is '^['.

login:

2. ftp(File Transfer Protocol):

Ftp is the user interface to the **Internet** standard File Transfer Protocol. The program allows a user to transfer files to and from remote network site. This utility helps you to upload and download your file from one computer to another computer.

The ftp utility has its own set of UNIX like commands which allow you to perform tasks such as:

- Connect and login to a remote host.
- Navigate directories.
- List directory contents
- Put and get files
- Transfer files as ascii, ebcdic or binary

Syntax: ftp hostname_or_ip

Example: ftp 172.16.1.254

Output: Connected to 172.16.1.254.

220 (vsFTPd 3.0.3)

Name (172.16.1.254:usha):

3. rlogin: rlogin command helps the user to log on to his identical remote account, without using either the username or password. The “*rlogin*” command is similar to telnet. The rlogin starts a terminal session on a remote host.

Syntax: rlogin hostname

Example: rlogin 172.16.1.254

Output: connect to address 172.16.1.254 port 513: Connection refused

Explanation:

Connects to a remote Unix system without password if configured.

If the service is disabled, connection is refused.

4) finger: The finger command is used to find out the details of a user, on both local and remote system. Finger may be disabled on other systems for security reasons.

Syntax: finger username

Example: finger usha

Output:

Login: usha Name:

Directory: /home/usha Shell: /bin/bash

On since Mon Aug 4 10:00 (IST) on pts/1 from 172.16.10.5

No mail.

No plan.

Filter

- A filter is a program that accepts input, transforms it and outputs the result.
- **Examples:** sort, cut, grep, tr, awk, sed.

Standard Input (STDIN):

- Default input source for a program is the keyboard.
- Symbolically represented as: 0
- Used when you type input manually.

Standard Output (STDOUT):

- Default output destination is the terminal screen.
- Symbolically represented as: 1
- Output from a command shows on your screen.

Redirect Output:

- Use > to redirect STDOUT to a file.
- **Example:** ls > files.txt
- (Output goes into files.txt instead of screen.)

• Redirect Input:

- Use < to redirect STDIN from a file.
- **Example:** sort < names.txt
- (Reads input from names.txt instead of keyboard.)

• Pipes (|):

- Connects output of one command to input of another.
- Sends STDOUT of command 1 as STDIN to command 2.
- **Example:** cat names.txt | sort | uniq
- (Reads from names.txt, sorts the lines, removes duplicates.)

cat Command in Linux:

- Display contents of a file on the terminal.
- Create a new file, Append data to existing files, Combine multiple files into one.
- Syntax: `cat [OPTIONS] [FILE]...`

Syntax: `cat -A file.txt` (-A → Show All Characters)

Ex: Hello<TAB>World

Output: Hello^IWorld\$

Syntax: `cat -b file.txt` (-b → Number Only Non-Empty Lines)

Adds line numbers but skips blank lines.

Ex: Input(file.txt):

Line 1

Line 2

Output:

1. Line 1

2. Line 2

Option	Meaning	Sample Usage	Notes
-A	Show all (-vET)	cat -A file.txt	Tabs = ^I, line ends = \$, etc.
-b	Number non-empty lines	cat -b file.txt	Skips numbering blank lines
-e	Show ends of lines + non-printables	cat -e file.txt	Adds \$ and displays control chars
-E	Show end of each line as \$	cat -E file.txt	Good for visualizing line breaks
-n	Number all lines	cat -n file.txt	Includes blank lines
-s	Squeeze multiple blank lines	cat -s file.txt	Reduces vertical space
-T	Show tabs as ^I	cat -T file.txt	Useful for checking tab characters
-v	Show non-printing characters	cat -v file.txt	Reveals control characters

1. To Create a new file: `cat > file1.txt`

2. To Append data into the file: `cat >> file1.txt`

3. To display a file: `cat file1.txt`

4. To concatenate several files and display: `cat file1.txt file2.txt`

The above cat command will concatenate the two files (file1.txt and file2.txt) and it will display the output in the screen. Some times the output may not fit the monitor screen. In such situation you can print those files in a new file or display the file using less command.

`cat file1.txt file2.txt | less`

5. To concatenate several files and to transfer the output to another file. `cat
file1.txt file2.txt > file3.txt`

head COMMAND: Display the Beginning of a File (Show first 10 lines (default)).

SYNTAX: \$ head [options] filename

Examples:

- head f1.txt : This command prints the first 10 lines of ' f1.txt '.
- head -5 f1.txt : The head command displays the first 5 lines of ' f1.txt '.
- head -c 5 f1.txt : The above command displays the first 5 characters of ' f1.txt '.

tail COMMAND: Display the End of a File(Show last 10 lines (default)).

SYNTAX: \$ tail [options] filename

EXAMPLE:

- tail index.php : It displays the last 10 lines of 'index.php'.
- tail -2 index.php : It displays the last 2 lines of 'index.php'.
- tail -n 5 index.php: It displays the last 5 lines of 'index.php'.
- tail -c 5 index.php : It displays the last 5 characters of 'index.php'.

cut COMMAND:

cut command is used to cut out selected fields of each line of a file. The cut command uses delimiters to determine where to split fields.

SYNTAX:

cut [options]

OPTIONS:

-c : Specifies character positions.

cut -c [position(s)] filename

-b : Specifies byte positions.

cut -b [position(s)] filename

-d : flags Specifies the delimiters and fields.

cut -d [delimiter] -f [field_numbers] filename

Example: If file.txt contains:

Orange	101,John,25
Banana	102,Alice,30
Grapes	103,David,28

Ex1: cut -c 1-3 file.txt

Ora

Ban

Gra

cut -b 1-5 file.txt

Orang

Banan

Grape

cut -d ',' -f 2,3 data.csv

john,25

Alice,30

David,28

paste COMMAND:

paste command is used to paste the content from one file to another file. It is also used to set column format for each line.

SYNTAX:

\$ paste [options]

OPTIONS:

- s Paste one file at a time instead of in parallel.
- d Reuse characters from LIST instead of TABs .

Example:

file1.txt	file2.txt
Red	Apple
Green	Leaf
Blue	Sky

Command1: paste file1.txt file2.txt

Command2: paste -d ':' file1.txt file2.txt

Command3: paste -s file1.txt

sort Command in Linux:

Sorts lines of text files in alphabetical or numerical order.

Syntax: sort [OPTIONS] filename

Options Description

- r Sort in reverse (descending) order
- u Remove duplicate lines (unique output)
- o Write the output to a file

Examples: Sample File: fruits.txt

Banana	1. Default Sort: sort fruits.txt
Apple	Output:
Mango	Apple
Apple	Apple
Cherry	Banana
banana	Cherry
	Mango
	banana
	(Note: Uppercase letters sort before lowercase.)

2. Reverse Sort (-r): sort -r fruits.txt

Output:

banana

Mango

Cherry

Banana

Apple

Apple

3. Unique Sort (-u): sort -u fruits.txt

Output:

Apple

Banana

Cherry

Mango

banana

Only one instance of each line is shown.

4. Output to File (-o): sort fruits.txt -o sorted_fruits.txt

This sorts fruits.txt and writes the sorted result into sorted_fruits.txt.

Combine Options: sort -ru fruits.txt

Output:

banana

Mango

Cherry

Banana

Apple

Sorted in reverse and duplicates removed.

tr COMMAND:

tr filter translates specified characters into other characters.

Normal usage: `tr string1 string2`

- Translates any character read from stdin that is in string1 into the corresponding character from string2; otherwise, the character is simply passed through.
- string1 and string2 normally have the same number of characters.

Eg `tr abc X5Z`

translates a into X, b into 5 and c into Z.

uniq Command:

- Removes consecutive duplicate lines from a file (or standard input).
- Important: The file should be sorted first for uniq to work as expected.
- Syntax: `uniq [OPTION] [INPUT_FILE] [OUTPUT_FILE]`

• Example File: **dups**

a
b
b
c
dd
ddd
ddd
Ddd

SNO	Example Command	Option Explanation	Output
1	<code>uniq dups</code>	Prints only one copy of each consecutive duplicate line.	a b c dd ddd Ddd
2	<code>uniq -u dups</code>	Prints only lines that occur exactly once in the entire file.	a c dd Ddd
3	<code>uniq -d dups</code>	Prints only lines that are repeated (one copy per group)	b ddd
4	<code>uniq -c dups</code>	Shows the number of times each line occurs	1 a 2 b 1 c 1 dd 2 ddd 1 Ddd

wc Command (Word Count):

Counts the number of lines, words, and characters in a file.

Syntax: wc [OPTION] filename

Example File :file1

Hello World

This is a test file

1	wc file1	3 7 40 file1
2	wc -l file1	3 file1
3	wc -w file1	7 file1
4	wc -c file1	40 file1
5	wc file1 file2	3 7 40 file1 5 12 68 file2 8 19 108 total

cmp COMMAND:

Compares two files and tells you what line numbers are different.

By default location of first mismatch is displayed

Syntax: \$ cmp [-options] filename1 filename2

-l: this option displays the list of all differences present in the file byte-by-byte.

-s: Write nothing for differing files; return exit status('0' or '1') only.

the '0' specifies that two files are similar and '1' specifies that there is at least one that is different.

file1.txt	file2.txt
Hello World Linux	Hello World Unix
cmp file1.txt file2.txt	Output: file1.txt file2.txt differ: byte 14, line 2
cmp -l file1.txt file2.txt	Output: 14 154 125 15 151 156
cmp -s file1.txt file2.txt echo \$?	Output: 1 0 → files are identical 1 → files differ 2 → error occurred

Text processing utilities

The operations performed by the text processing utilities are as follows

Processing the text, Modifying the content of files, Deleting words and characters from files

cat: cat is used to create the files.

```
$cat> filename
```

Type some text here

Press ctrl+d

Cat can also be used to display the contents Of a file.

```
$cat filename
```

Cat can also concatenate the contents of 2 files and store them in third file.

```
Cat>file1 file2>new file
```

To append the contents of two files into another file use

```
Cat>file1 file2>>new file
```

tail: tail command displays the end of the file. It displays the last ten lines by default.

syntax: tail [OPTION]... [FILE]

To display last 3 lines use

`$tail -n 3 file` (or) `$tail -3 file`

We can also address the lines from the beginning of the file instead of the end.

The + count allows to do that.

head: head command as the name implies, displays the top of the file. When used without an option, it displays the first 10 lines of the file.

`$head file`

We can use `-n` option to specify a line count and display, say first 3 lines of the file.

`$head -n 3 file` or `$head -3 file`

Sort: Sort can be used for sorting the contents of a file.

`$sort shortlist`

Sorting starts with the first character of each line and proceeds to the next character only when the characters in two lines are identical.

Sort options:

-t:	Specify a delimiter to separate fields in each line. Field numbers start from 0 in old syntax (+n), but in modern syntax use -k.	sort -t "delimiter" +<field_number>	\$ sort -t " " +2 shortlist # Sorts based on the 3rd field (marks) 101 Alice 85 102 Bob 88 103 John 90
-r	Sort results in descending order.	sort -r shortlist	grape banana apple
You can sort by multiple fields : Primary key: first sort criterion Secondary key: used when the primary key matches.			
	Primary key: 3rd field (marks) Secondary key: 2nd field (name) This ensures tie-breaking if marks are the same.	\$ sort -t " " +2 -3 +1 shortlist	104 John 75 101 Alice 85 103 Bob 85 105 Charlie 85 102 Alice 90

nl(number lines):

- It is used to automatically number the lines of a file.
- It numbers **logical lines** (lines that contain some content).
- It **does not** number empty lines by default.

File (sample.txt):	nl sample.txt	nl -s: sample.txt
Hello World Linux	1 Hello 2 World 3 Linux	1:Hello 2:World 3:Linux

-n (grep) → Shows the line numbers where the pattern occurs.

-c (grep) → Shows only the total number of lines containing the pattern.

File (sample.txt):	grep -n "World" sample.txt	grep -c "World" sample.txt
Hello World Linux World	2:World 4:World	2

Grep:

- globally search for a regular expression and print.
- Grep scans a file for the occurrence of a pattern and depending on the options used, displays
- Lines containing the selected pattern.
- Lines not containing the selected pattern (-v).
- Line numbers where pattern occurs (-n)
- No. of lines containing the pattern (-c)

option (lowercase L): -l

Shows only the filenames where the pattern is found, not the matching lines.

Example: `grep -l "error" *.log`

This will print the names of all .log files containing the word "error".

2. egrep (extended grep):

- Works like grep but supports extended regular expressions without needing to escape special characters.
- Supports special regex symbols like:
 - + → matches one or more occurrences of the previous character.
 - ? → matches zero or one occurrence of the previous character.

Example: `egrep "colou?r" file.txt`

Matches both "color" and "colour".

3. fgrep (fast grep):

- Faster because it does not interpret regular expressions.
- Only searches for fixed strings (literal text).
- Best for searching special characters like ?, +, * without escaping them.

Example: `fgrep "?" file.txt`

Finds all lines containing the ? character literally.

Cut: slitting the file vertically

U can slice a file vertically with cut command.

Cutting columns(-c):specifies character positions to extract.

<range> can be:

1-4 → columns 1 through 4

1,3,5 → specific columns

3- → from column 3 to the end

Example: Suppose group1 contains:

root:x:0:root

bin:x:1:bin

guest:x:100:guest

Command: cut -c 1-4 group1

Root

bin

gues

cut -d: -f1,3 group1

root:0

bin:1

guest:100

Paste: pasting files

What u cut with the cut can be pasted back with paste command-but vertically rather than horizontally. u can view two files side by side by pasting them.

To join two files calc.lst and result.lst use

```
$paste -d= calc.lst result.lst
```

File: calc.lst	File: result.lst	paste -d= calc.lst result.lst
10+5 20-7 3*4	15 13 12	10+5=15 20-7=13 3*4=12

Join:

The join command in Unix/Linux is used to merge lines from two sorted text files based on a common field (usually the first field unless specified otherwise).

How it works

- Both files should be sorted on the join field.
- By default, the first field in each file is considered the common field.
- The output combines the common field once, followed by the remaining fields from both files.

Syntax: join [OPTION] file1 file2

File1 (file1.txt)	File2 (file2.txt)	join file1.txt file2.txt
1 apple 2 banana 3 cherry	1 red 2 yellow 3 red	1 apple red 2 banana yellow 3 cherry red

Back up utilities

cpio (Copy In/Out):

Copy files **to and from** a backup device (e.g., tape, file, another directory).

How it works:

- Reads the list of filenames from **standard input**.
- Copies the files (including their headers) into a continuous data stream.
- This stream can be **redirected** to a file or device, or piped to another command.

Main options:

- -o → Output mode (create an archive).
- -i → Input mode (extract from an archive).

Example:

```
ls | cpio -ov > backup.cpio    # Create backup from file list
```

```
cpio -iv < backup.cpio        # Extract backup
```

```
find . -iname "*.txt" | cpio -ov > sample.cpio #All .txt files are archived into sample.cpio
```

tar (Tape Archive):

- Store and extract multiple files/directories into/from a single archive file.
- Takes file and directory names as arguments (no need to pipe a list like cpio).
- Recursive by default — includes subdirectories.
- Allows multiple versions of the same file in one archive.
- Can append to an existing archive without overwriting it.

Common options:

- c → Create archive.
- v → Verbose output (shows files processed).
- f → Specify filename/device for archive.
- x → Extract from archive.

Examples:

Command	Purpose
<code>tar -cvf countries.tar countries/</code>	Create archive from directory
<code>tar -tvf countries.tar</code>	List contents of archive
<code>tar -rvf countries.tar newfile</code>	Append new file to archive
<code>tar -xvf countries.tar -C basics/</code>	Extract archive into basics/

sed (Stream Editor):

- A Linux command-line tool used for text processing.
- It reads input line by line, applies editing commands, and outputs the result — without changing the original file unless explicitly told.

Operations

Substitution (s) → Replace text.

Deletion (d) → Remove matching lines.

Printing (p) → Show specific lines.

Insertion (i) → Add text before a line.

Appending (a) → Add text after a line.

Change (c) → Replace the whole line.

Addresses in sed

You can target specific lines or patterns.

By line number: `sed '3d' file.txt` *# Delete line 3*

By pattern: `sed '/error/d' file.txt` *# Delete lines containing "error"*

Range: `sed '5,10d' file.txt` *# Delete lines 5 to 10*

`sed '/start/,/end/d' file.txt` *# Delete from "start" to "end"*

Sed Options:

- -e : Script is given directly on command line
- -f : Script is taken from a separate file
- -n : Suppresses automatic printing, output only with p command

Addresses in Sed

Range Address:

- start,end format
- 4,9d → delete lines 4 to 9
- 2,/A/p → from line 2 to line starting with A, print

Set-of-Lines Address:

- Use regular expressions
- /A/p → print lines starting with A
- /^\$/d → delete blank lines

Examples:

`sed -n '3p' input.txt` → show line 3

`sed -n '$p' input.txt` → show last line

`sed '10s/endif/fi/' input.txt` → replace on line 10

`sed '/^$/d' input.txt` → delete blank lines

Sed Editing Commands:

`=` → print line numbers

`i` → insert before line

`c` → change/replace line

`a` → append after line

`d` → delete line(s) by number or pattern

`sed '=' twister.dat`

`sed '2i\Inserted Line' twister.dat`

`sed -f script.sed twister.dat`

`sed -f script.sed twister.dat`

`sed '/^[BI]/d' twister.dat`

AWK

- **awk** is a powerful text-processing language.
- It was created by **Aho, Weinberger, and Kernighan** (hence the name **AWK**).
- It reads input **line by line**, splits each line into **fields**, and processes them according to **instructions (scripts)**.

Syntax: `$ awk [options] 'script' file(s)`

Options:

1. **-f scriptfile** : Runs awk program from a script file.

`$ awk -f myscript.awk data.txt`

2. **-F separator**: Defines input field separator(Default: space or tab).

`$ awk -F "," '{print $1,$3}' students.csv`

Data Handling in awk:

File = Collection of Records

Record = Line of data

Fields = Units of data within a line

By default: Record separator (RS) = newline (\n)

Field separator (FS) = whitespace (space or tab)

Field Variables in awk: \$0 → entire line (record)

\$1, \$2, \$3... → fields in the current line

Example file (info.dat):

101 John 85

102 Mary 90

103 David 78

\$ awk '{print \$1, \$2}' info.dat

101 John

102 Mary

103 David

\$ awk '{print \$0}' info.dat

101 John 85

102 Mary 90

103 David 78

Field Buffers in awk:

When awk reads a line (record), it splits it into fields.

Each field is stored in a buffer (temporary storage).

A field buffer is identified by \$ followed by the field number.

Rules:

\$0 → entire record (whole line).

\$1 → first field.

\$2 → second field.

\$3 → third field ... up to \$NF (last field).

Number of field buffers = NF (Number of Fields in that record).

Example:

Ex1: `$ awk '{print $1,$3}' students.txt`

Student.txt

101 John 85 CSE
102 Mary 90 ECE
103 David 78 ME

101 85
102 90
103 78

Ex2: `$ awk '{print $0}' students.txt`

101 John 85 CSE
102 Mary 90 ECE
103 David 78 ME

Important Built-in Variables(System Variables):

FS → Input Field Separator(tab-separated file):

```
$ awk -F "\t" '{print $1, $3}' info.dat
```

RS → Input Record Separator

Default: newline.

Example (reading records separated by .): `$ awk 'BEGIN{RS="."} {print $0}' paragraph.txt`

OFS → Output Field Separator

```
$ awk 'BEGIN{OFS=" - "} {print $1,$2}' info.dat
```

Output:

101 - John

102 - Mary

103 - David

ORS → Output Record Separator

```
$ awk 'BEGIN{ORS="\n---\n"} {print $1,$2}' info.dat
```

Output:

```
101 John
---
102 Mary
---
103 David
---
```

NF → Number of Fields in the current record

```
$ awk '{print NF}' info.dat
```



Output:

```
3
3
3
```

NR → Number of records read from all files

```
$ awk '{print NR,$0}' info.dat
```

Output:

```
1 101 John 85
2 102 Mary 90
3 103 David 78
```

FNR → Number of records in current file

```
$ awk '{print FNR,$0}' info1.dat info2.dat
```

```
(info1.dat)
1 101 John 85
2 102 Mary 90
```

...

```
(info2.dat)
1 201 Alex 88
2 202 Ruby 92
```

Ex:

```
$ awk '{print $NF}' info.dat
```

```
85
90
78
```

awk script : An awk script can have three sections:

❑ **BEGIN block (pre-processing):**

Runs before input file processing starts.

Used for initialization tasks (e.g., printing headers, setting variables).

❑ **BODY block (processing):**

Runs once for each input record.

Contains logic that operates on records.

❑ **END block (post-processing)**

Runs after all records are processed.

Used for summary or final results.

Operations:

Example: Calculate Total Marks

File: records.dat

```
1 A 75 81 89
2 B 86 94 82
3 C 90 71 96
4 D 82 70 85
```

Script: total.awk

BEGIN section → Initialization

```
BEGIN {print "Name\tTotal"}
```

BODY section → Runs once for each record

```
{ total = $3 + $4 + $5
  print $2 "\t" total }
```

END section → Final summary

```
END {print "Best Result"}
```

RUN command:

```
$ awk -f total.awk records.dat
```

OUTPUT:

Name	Total
A	245
B	262
C	257
D	237

Best Result

Pattern/Action:

- Awk can process only records that match a condition.
- Matching is done using regular expressions inside /.../.

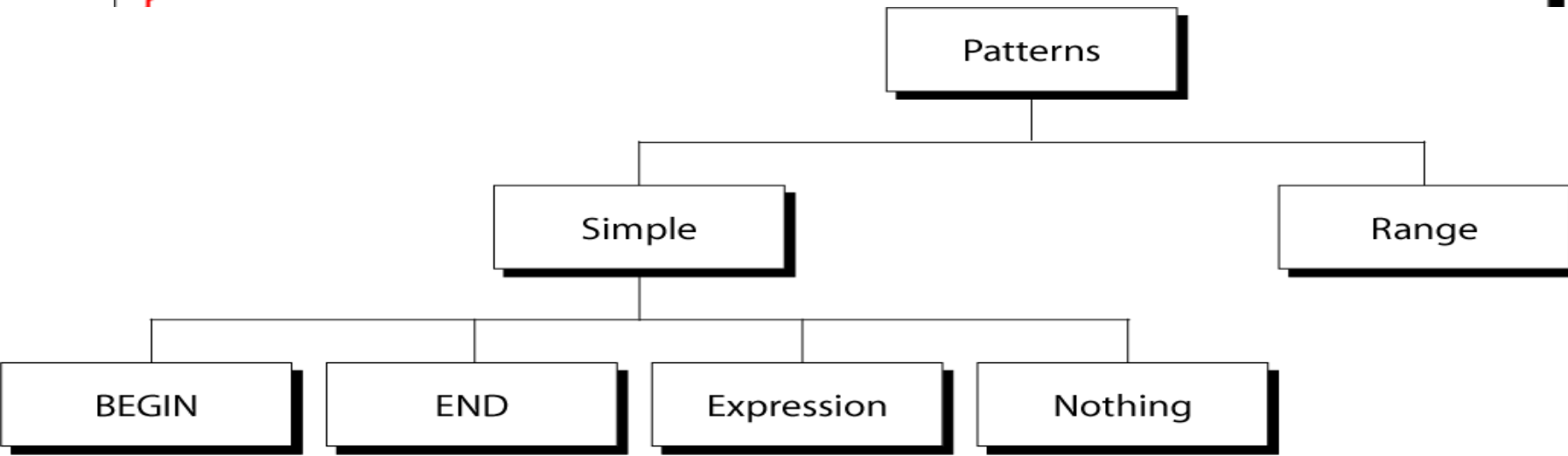
```
pattern {statement}
```

(a) One Statement Action

```
pattern {statement1; statement2; statement3}
```

(b) Multiple Statements Separated by Semicolons

```
pattern  
{  
    statement1  
    statement2  
    statement3  
}
```



Example 1: Match records ending with 00

File: employees2

Tom Jones:4424:5/12/66:543354
Mary Adams:5346:11/4/63:28765
Sally Chang:1654:7/22/54:650000
Billy Black:1683:9/23/44:336500

\$ awk -F: '/00\$/' employees2

-F: sets : as field separator.

/00\$/ matches lines ending with "00".

Output:

Sally Chang:1654:7/22/54:650000
Billy Black:1683:9/23/44:336500

5. Pattern Matching Operators

~ → match

!~ → not match

Example:

\$ awk -F: '\$2 ~ /16/' employees2

Prints records where 2nd field contains 16.

Example 1: Match a word in entire record

File: emps.txt

Tom Jones 4424 5/12/66 543354

Mary Adams 5346 11/4/63 28765

Sally Chang 1654 7/22/54 650000

Billy Black 1683 9/23/44 336500

Command: `$ awk '/Mary/ {print $0}' emps.txt`

Output:

Mary Adams 5346 11/4/63 28765

Example 2: Match lines starting with "Tom"

Command: `$ awk '/^Tom/ {print $0}' emps.txt`

Output:

Tom Jones 4424 5/12/66 543354

Example 4: Match lines containing digits

Command: `$ awk '/[0-9]+/ {print $0}' emps.txt`



Output:

Tom Jones 4424 5/12/66 543354

Mary Adams 5346 11/4/63 28765

Sally Chang 1654 7/22/54 650000

Billy Black 1683 9/23/44 336500

awk Operators, Variables, and Control Structures

Operator	Meaning	Example
+	Add	$x + y$
-	Subtract	$x - y$
*	Multiply	$x * y$
/	Divide	x / y
%	Modulus	$x \% y$
^	Exponential	$x ^ y$

File: nums.txt

10 20
5 30
15 40

COMMAND:

\$ awk '\$1 * \$2 > 200 {print \$0}' nums.txt

Output:

15 40

2. Relational Operators

Operator	Meaning	Example
<	Less than	$x < y$
<=	Less than or equal to	$x \leq y$
==	Equal to	$x == y$
!=	Not equal to	$x != y$
>	Greater than	$x > y$
>=	Greater than or equal to	$x \geq y$
~	Matches regex	$\$1 \sim /Tom/$
!~	Not matched by regex	$\$1 !\sim /Tom/$

Example

File: marks.txt

A 45

B 75

C 90

command:

```
$ awk '$2 >= 60 {print $1,"Pass"}' marks.txt
```

Output:

B Pass

C Pass

3. Logical Operators

Operator	Meaning	Example
&&	Logical AND	a && b
	Logical OR	a b
!	NOT	!a

Example 1: AND

Command: `$ awk '($2 > 50) && ($2 < 90) {print $0}' marks.txt`

Output:

B 75

Example 2: OR

Command: `$ awk '$2 == 90 || $1 == "A" {print $0}' marks.txt`

Output:

A 45

C 90

4. Range Patterns:

Syntax: pattern1, pattern2 {action}

Awk prints all lines between pattern1 and pattern2 (inclusive).

Example: File: emps

Tom

Mary

Sally

Billy

John

Command: `$ awk '/Mary/,/Billy/ {print $0}' emps`

Output:

Mary

Sally

Billy

5. Expressions:

- Expressions = combination of variables, constants, operators, regex, or functions.
- They can be used in conditions, assignments, or print statements.

Example: File: emps

Tom Jones 4424 50
Mary Adams 5346 60
Sally Chang 1654 45
Billy Black 1683 55

Command:

```
$ awk '$1 ~ /Tom/ {wage = $3 * $4; print wage}' employees.txt
```

Fields in first line:

\$1 = Tom
\$2 = Jones
\$3 = 4424
\$4 = 50

Since \$1 is "Tom", condition is true.

Wage = 4424 * 50 = 221200.

6. Variables:

- User-defined variables can be created anytime.
- They don't need declaration; they are initialized to empty string ("") or 0.
- Can be numbers, strings, or arrays.

Syntax: variable = expression

Example 1: Assign variable

Command1: `$ awk '$4 == "CA" {$4="California"; print $0}' empdata.txt`

If 4th field = CA, replace with "California".

Example 2: Computed variable

Command2: `$ awk '{total=$3+$4; print $1,total}' records.dat`

7. Assignment Operators

Operator	Meaning
=	Assign
++	Increment by 1
--	Decrement by 1
+=	Add & assign
-=	Subtract & assign
*=	Multiply & assign
/=	Divide & assign
%=	Modulus & assign
^=	Exponentiation & assign

Example: `$ awk '{count++; sum+= $2} END {print "Count=",count,"Sum=",sum}' marks.txt`

8. Output Statements:

marks.txt

Alice 85
Bob 92
Charlie 78
David 88

1. print – simple output.

`$ awk '{print $1,$2}' marks.txt`

2. printf – formatted output (like C's printf).

`$ awk '{printf "%-10s %3d\n",$1,$2}' marks.txt`

3. sprintf – creates a formatted string (stored in variable).

`{line = sprintf("%s: %d",$1,$2); print line}`

9. Conditional Statements (if-else)

Syntax:

```
if (condition)
    statement1
else
    statement2
```

Example:

```
$ awk '{if($2>=60) print $1,"Pass"; else print $1,"Fail"}' marks.txt
```

Output:

```
A Fail
B Pass
C Pass
```

10. Loops: for (like C loop)

```
for (init; condition; update)
    statement
```

Example:

```
for (i=1; i<=NF; i++)
    print i, $i
```

for-in (for arrays):

```
for (var in array)
    statement
```

Example:

```
for (dept in sales)
    print dept, sales[dept]
```

while loop:

```
while (condition)
    statement
```

Example:

```
i=1
while (i<=NF) { print $i; i++ }
```

do-while loop:

```
do {
    statement
} while (condition)
```

```
i = 1
do {
    print $0
    i++
} while (i <= 10)
```

11. Loop Control Statements

break → exit loop.

continue → skip current iteration.